A VAR-based Computational Analysis of Influenza and Weather Dynamics

Rohan Adla, Arrio Gonsalves, Shreyan Nalwad, Dylan Setiawan

December 13, 2025

Abstract

Contents

1 Introduction **3**
1.1 Biological Motivation and Problem Statement . . . 3
1.2 From Biological Questions to Computational Problems . . . 3
1.3 Overview of Approach . . . 4

2 Technical Background **4**
2.1 Time Series Analysis in Epidemiology . . . 4
2.2 Vector Autoregression (VAR) . . . 5
2.3 Impulse Response Functions (IRFs) . . . 5
2.4 Granger Causality . . . 6
2.5 Bootstrap Methods for Time Series . . . 6

3 Data and Preprocessing **7**
3.1 Datasets . . . 7
3.2 Preprocessing and Feature Engineering . . . 7
3.3 Assumption Checking and Model Diagnostics . . . 8
3.4 Final Variable Set . . . 10

4 Algorithms and Implementation **10**
4.1 Software Architecture . . . 11
4.2 VAR Estimation via OLS . . . 13
4.3 Impulse Response Computation . . . 14
4.4 Granger Causality Testing . . . 15
4.5 Residual Bootstrapping for IRFs . . . 15
4.6 Residual Bootstrapping for Granger Causality . . . 16

5 Results **16**
5.1 Model Fit and Forecasting Performance . . . 17
5.2 Impulse Response Analysis . . . 17
5.3 Granger Causality Results . . . 18
5.4 Bootstrap Uncertainty Estimates . . . 18

6 Discussion and Conclusion **19**

7 Acknowledgments **20**

A Supplementary Methods: Mathematical Details **21**
A.1 Vector Autoregression (VAR) . . . 21
A.1.1 Model specification . . . 21
A.1.2 Deterministic regressors . . . 21
A.1.3 Stacked regression form . . . 21
A.2 Estimation via Ordinary Least Squares . . . 22
A.3 Moving-Average Representation and IRFs . . . 23
A.3.1 Companion form . . . 23
A.3.2 Infinite-order moving-average representation . . . 24
A.3.3 Structural shocks and identification . . . 24
A.3.4 Impulse response functions . . . 25
A.4 Granger Causality Test Statistic . . . 25
A.4.1 Definition . . . 25
A.4.2 F-statistic for linear restrictions . . . 26
A.5 Residual Bootstrap for IRFs . . . 26
A.5.1 Algorithm . . . 27
A.6 Residual Bootstrap for Granger Causality . . . 28

B Author Contributions **29**

1 Introduction

1.1 Biological Motivation and Problem Statement

Seasonal influenza is a recurrent infectious disease that poses a substantial burden on global public health. Its transmission dynamics are influenced by a combination of biological, environmental, and behavioral factors, including viral characteristics, host susceptibility, seasonal patterns, and local climate conditions. Despite extensive surveillance efforts, accurately forecasting influenza activity and understanding the drivers of its temporal variation remain challenging.

From a biological perspective, influenza incidence exhibits strong temporal structure: cases tend to rise and fall in seasonal waves, with marked differences across geographic regions. Environmental factors such as temperature and humidity have been shown to affect viral survival, transmissibility, and host susceptibility, while interactions between influenza A and B strains may further shape observed incidence patterns. Understanding these temporal dependencies is critical for improving outbreak forecasting, informing public health interventions, and supporting policy decisions such as vaccination timing and resource allocation.

The central scientific question motivating this project is therefore: *how do influenza dynamics evolve over time, and to what extent can past influenza activity and weather conditions explain and predict future incidence?* Addressing this question requires models that can capture both temporal dependence and interactions among multiple time-varying factors.

1.2 From Biological Questions to Computational Problems

The biological questions outlined above naturally translate into a set of computational time-series problems. First, influenza forecasting can be framed as a multivariate prediction task, where future influenza activity is predicted using past observations of influenza incidence and environmental covariates. Second, the influence of weather variables on influenza dynamics raises causal questions: specifically, whether changes in temperature or humidity provide statistically significant predictive information about future influenza activity beyond what is contained in past influenza values alone. These questions can be formalized using Granger causality tests.

In addition, epidemiological interest often extends beyond prediction to understanding system responses to external perturbations. For example, how does a sudden drop in temperature affect influenza incidence over subsequent weeks? Such questions can be addressed through impulse response analysis, which quantifies the dynamic effect of a one-time shock to one variable on the future trajectory of others.

Together, these considerations motivate a modeling framework that supports:

- Forecasting influenza dynamics using past influenza and weather data,
- Testing whether weather variables Granger-cause influenza incidence,
- Quantifying the temporal response of influenza activity to exogenous shocks.

1.3 Overview of Approach

To address these goals, we adopt a Vector Autoregression (VAR) framework, which models multiple time series jointly as functions of their own past values and the past values of other variables in the system. Our computational pipeline proceeds as follows. First, we preprocess and clean influenza surveillance data and country-level weather data, aligning them temporally and constructing features suitable for time-series modeling. Second, we fit our VAR(p) model with lagged p values on country-specific datasets. Third, we compute impulse response functions (IRFs) to analyze the dynamic effects of weather shocks on influenza activity. Fourth, we perform Granger causality tests to assess directional predictive relationships between variables. Finally, uncertainty in IRFs and causality statistics is quantified using resampling-based methods.

The analysis is implemented using a custom computational pipeline, with data preprocessing and exploratory analysis performed in Python, core VAR, Granger causality, and IRF computations implemented in a Go backend for efficiency, and visualization and interaction supported via a Shiny-based front-end.

2 Technical Background

Our exposition of vector autoregressions, impulse response functions, and bootstrap-based inference follows Hamilton [1] and Kilian [2].

2.1 Time Series Analysis in Epidemiology

Time series models are widely used in epidemiology to understand how diseases evolve over time and how external factors, such as weather or mobility patterns, influence transmission. A time series is a sequence of observations collected at regular intervals. In a *univariate* time series, only one variable (e.g., weekly influenza cases) is modeled, while a *multivariate* time series considers several variables simultaneously (e.g., influenza counts, temperature, humidity, and precipitation). Multivariate approaches are essential in epidemiological contexts because disease dynamics rarely depend on a single quantity; instead, multiple environmental and behavioral factors interact to shape outbreak patterns.

Several features make epidemiological time series challenging. First, many processes exhibit strong

lagged effects: for example, changes in humidity may influence influenza transmission several days or weeks later. Second, epidemiological data often display substantial *seasonality*, meaning the statistical properties of the series change periodically (e.g., winter peaks in influenza). Finally, different variables may exert *mutual and time-delayed influence* on one another, which cannot be captured adequately by a collection of separate univariate models. These considerations motivate the use of multivariate frameworks that model all variables jointly and account for their temporal dependencies.

Vector Autoregression (VAR) provides a flexible and interpretable framework for doing so. VAR models treat all variables symmetrically, allowing each to depend on past values of itself and of all other variables in the system. This approach naturally captures feedback loops, delayed effects, and dynamic interdependencies, all of which are central in epidemiological analysis.

2.2 Vector Autoregression (VAR)

The VAR(p) model is a multivariate extension of the standard autoregressive model. At a high level, the idea is: at each time point t, the vector of variables $\mathbf{y}_t \in \mathbb{R}^K$ is predicted using p previous observations:

$$\mathbf{y}_t \approx f(\mathbf{y}_{t-1}, \mathbf{y}_{t-2}, \ldots, \mathbf{y}_{t-p}).$$

Unlike univariate autoregressive models, the VAR allows each variable to depend on the lagged values of *all* variables. This symmetry is valuable when the direction and strength of interactions are uncertain—for example, when exploring how much of influenza variability is explained by past weather versus its own past trajectory.

We use a *reduced-form VAR*, in which all variables appear on the left-hand side, and the right-hand side contains only lagged variables and deterministic terms (e.g., intercept, seasonal regressors). This form is convenient for estimation because each equation can be fit with ordinary least squares, making the approach computationally efficient and robust. Though the reduced-form coefficients do not directly represent structural causal effects, they provide a consistent and interpretable framework for uncovering predictive relationships. Structural interpretation is handled later through impulse response functions.

2.3 Impulse Response Functions (IRFs)

A key advantage of VAR modeling is the ability to characterize the *dynamic effect* of one variable on another through impulse response functions (IRFs). An IRF traces the time path of the system following a one-time, unexpected shock to one variable. For example, an IRF can reveal how a sudden increase in temperature affects influenza incidence over the next several weeks.

Intuitively, IRFs answer questions of the form:

> "If we nudge variable j upward today by a small amount, how will variable k respond over time?"

This is especially useful in epidemiology, where interventions or environmental events (such as abrupt weather changes) may have delayed ripple effects. IRFs allow us to quantify the magnitude, direction, and persistence of these effects. They also help distinguish immediate responses from long-run adjustments, enabling more informed interpretation of complex disease-environment interactions.

To compute IRFs, we convert the VAR into its moving-average representation and apply a carefully defined shock to one variable. Because variables in the reduced-form VAR are typically correlated, shocks must be *orthogonalized* (e.g., via a Cholesky decomposition) to avoid confounding responses across variables. The detailed derivation of IRFs is provided in Appendix A.3.

2.4 Granger Causality

Granger causality offers a formal way to test whether one time series improves the prediction of another. The idea is *predictive* rather than mechanistic: a variable X Granger-causes a variable Y if past values of X contain information that helps predict Y beyond what is contained in the past values of Y itself and any other included variables.

For example, if including lagged humidity values reduces the forecast error for influenza incidence, then humidity *Granger-causes* influenza in the predictive sense. Importantly, Granger causality does not claim biological or mechanistic causation; instead, it identifies directional predictive relationships, which can guide further epidemiological investigation.

In the context of a VAR, Granger causality tests reduce to comparing two regression models for a target variable: one that includes the lagged values of a candidate predictor and one that excludes them. A statistically significant reduction in residual variance when the predictor is included indicates Granger causality. The mathematics underlying these F-tests is described in Appendix A.4.

2.5 Bootstrap Methods for Time Series

Although classical asymptotic results provide approximate standard errors and confidence intervals for VAR-based statistics, these approximations can perform poorly in small samples or when model assumptions (e.g., normal residuals) are violated. Epidemiological datasets are often limited in length or contain structural breaks, making purely analytical uncertainty quantification unreliable.

To address this, we adopt a *residual bootstrap* approach. After estimating the VAR, we resample the residuals, generate synthetic datasets under the fitted model, re-estimate the VAR parameters,

and recompute the IRFs or Granger statistics for each bootstrap sample. Repeating this process many times yields an empirical distribution that captures model uncertainty in a more data-driven and assumption-light manner.

The bootstrap provides:

- **Confidence intervals for IRFs**, describing the range of plausible dynamic responses.
- **Bootstrap p-values for Granger causality**, which can be more stable than asymptotic tests when samples are short.

This approach ensures that our conclusions about influenza–weather relationships are robust and not overly dependent on restrictive analytical assumptions. Full details of the bootstrap algorithm are given in Appendices A.5 and A.6.

3 Data and Preprocessing

3.1 Datasets

The primary epidemiological data used in this study comes from the World Health Organization's Global Influenza Surveillance system, FluNet [8]. This dataset provides weekly, country-level records of laboratory-confirmed influenza detections, including counts of influenza A and B, as well as associated surveillance metadata, spanning multiple influenza seasons and enabling the analysis of both short-term dynamics and longer-term seasonal patterns.

To capture environmental drivers of influenza transmission, we integrate country-level weekly weather data from the National Oceanographic and Atmospheric Administration's (NOAA) Global Historical Climatology Network (GHCN) hourly dataset [9]. The weather dataset includes variables such as average temperature, dew point temperature or relative humidity, wind speed, atmospheric pressure, visibility, wet-bulb temperature, and multiple measures of precipitation, which are known to influence influenza virus viability, transmission, and host susceptibility.

3.2 Preprocessing and Feature Engineering

All datasets are aligned to a common weekly time scale using ISO year and ISO week identifiers to ensure consistency across variables. FluNet records are first filtered by country code (e.g., `SGP` for Singapore) and then restricted to rows with valid numeric ISO year and week values before being sorted by these keys. Weather records are similarly filtered to retain only rows with valid ISO identifiers, and observations are then aggregated so that there is exactly one weather record per (`iso_year`, `iso_week`) pair.

From the raw weather feed, we remove columns that are either static metadata (e.g., station

identifiers, station names, latitude, longitude, elevation), highly sparse (e.g., short-horizon pressure change, wind gusts), or redundant with other variables (e.g., station-level versus sea-level pressure, overlapping precipitation horizons). The remaining meteorological predictors include temperature, dew point temperature, relative humidity, wind speed, sea level pressure, visibility, wet-bulb temperature, and both instantaneous and 24-hour cumulative precipitation. For these continuous weather time series, missing values are imputed using linear interpolation over time, which fills small gaps while preserving the weekly temporal structure.

After preprocessing, the influenza and weather datasets are merged on (`iso_year`, `iso_week`) using an inner join, ensuring that only weeks with both valid influenza and weather observations are retained. All explicit date and identifier columns (such as week start dates, MMWR identifiers, and country codes) are then dropped from the merged table so that the remaining columns correspond only to influenza outcomes and weather covariates. To stabilize variance and reduce the influence of large epidemic peaks, influenza A and B case counts are transformed using a log-plus-one mapping, producing `LOG_INF_A` and `LOG_INF_B` as the primary response variables.

For each country, we construct two multivariate time series: one pairing `LOG_INF_A` with the shared weather covariates, and another pairing `LOG_INF_B` with the same set of predictors. These strain-specific design matrices are then subjected to a set of assumption checks prior to VAR estimation. Augmented Dickey–Fuller and KPSS tests are applied to assess stationarity of each series, a RESET-type regression test is used to screen for deviations from linearity over time, and variance inflation factors are computed to diagnose multicollinearity among covariates. For series that are non-stationary or non-linear and have nonnegative support, additional log-transformed versions of the corresponding weather covariates (e.g., `LOG_PRECIPITATION`, `LOG_VISIBILITY`) are created and re-evaluated to confirm improved behavior with respect to these assumptions. The final, assumption-checked matrices are exported as country- and strain-specific CSV files and form the inputs to the downstream VAR, impulse response, and Granger causality analyses.

3.3 Assumption Checking and Model Diagnostics

Before estimating our VAR model, we evaluated whether the final country-specific datasets reasonably satisfy the statistical assumptions underlying linear multivariate time-series analysis (Lütkepohl, 2005). All diagnostics were implemented programmatically in Python using the `statsmodels` library.

Stationarity

VAR models require all endogenous variables to be weakly stationary, meaning that their mean and variance remain stable over time (Hamilton, 1994). We assessed stationarity using the Augmented Dickey–Fuller (ADF) test, which evaluates the null hypothesis of a unit root in the autoregressive

representation

$$x_t = \rho x_{t-1} + \sum_{i=1}^{k} \gamma_i \Delta x_{t-i} + \varepsilon_t.$$

Failure to reject the null hypothesis $\rho = 1$ indicates non-stationarity. Raw influenza counts strongly violated this assumption, motivating a logarithmic transformation followed by first differencing:

$$\Delta \log(1 + x_t) = \log(1 + x_t) - \log(1 + x_{t-1}).$$

This transformation converts level counts into approximate weekly growth rates and substantially improved stationarity properties in subsequent ADF tests.

Linearity

VAR models assume approximately linear relationships between variables and their lagged values (Lütkepohl, 2005). To assess potential nonlinear structure, we applied RESET-style specification tests, which examine whether nonlinear functions of fitted values significantly improve model fit. While mild nonlinearities remained—particularly during periods of rapid epidemic growth—these deviations were substantially reduced after transformation and were deemed acceptable for linear autoregressive modeling.

Multicollinearity

High multicollinearity among regressors can lead to unstable coefficient estimates and inflated standard errors (Hamilton, 1994). We quantified multicollinearity using Variance Inflation Factors (VIFs), defined as

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2},$$

where R_j^2 is obtained by regressing variable j on all other variables. Variables that were exact or near-exact linear combinations of others—such as total influenza counts derived from strain-specific counts or proportion variables summing to one—exhibited infinite or extremely large VIFs and were removed prior to modeling.

Homoscedasticity

Classical VAR estimation assumes homoscedastic error terms; however, epidemiological time series frequently exhibit time-varying variance due to outbreak dynamics. We assessed variance stability using ARCH tests (Engle, 1982), which test the null hypothesis

$$\mathrm{Var}(\varepsilon_t \mid \mathcal{F}_{t-1}) = \sigma^2.$$

Consistent with prior literature, evidence of heteroscedasticity persisted in influenza growth rates and some weather variables. Rather than attempting to fully eliminate this behavior, we acknowl-

edge heteroscedasticity as an intrinsic property of epidemic data.

Lag Order Selection

The choice of lag order p determines how many past observations are used to predict current values and is a critical modeling decision in VAR analysis [3]. In this project, we selected the lag order using a combination of domain knowledge and exploratory model diagnostics. Because influenza transmission and weather effects often unfold over several weeks, we chose a lag length that captures these biologically meaningful delays while maintaining numerical stability and interpretability. The selected lag order was then held fixed across all estimation, IRF computation, and bootstrap procedures to ensure consistent comparisons across countries and variants.

Summary of Diagnostics

Overall, these diagnostics indicate that, after preprocessing and feature engineering, the data satisfy the core assumptions required for VAR modeling to a reasonable degree. Remaining departures—particularly heteroscedasticity and mild nonlinearities—are consistent with the biological and seasonal nature of influenza transmission and are commonly encountered in applied epidemiological time-series analysis (Shaman and Kohn, 2010).

3.4 Final Variable Set

After preprocessing and feature engineering, the final multivariate time series used for VAR modeling for each country consists of a log-transformed influenza outcome together with core weekly weather covariates and, where beneficial, additional log-transformed weather features. Table 1 summarizes the key variables that enter the country- and strain-specific VAR training datasets.

4 Algorithms and Implementation

In this section we describe how the core algorithms—VAR estimation, impulse response computation, Granger causality testing, and residual bootstrapping— are implemented in our software stack. Our design goals were: (i) to keep the numerical routines as close as possible to the mathematical specification in Appendix A, (ii) to make the code modular so that additional models or estimators can be plugged in later without major refactoring, and (iii) to exploit parallelism for computationally expensive tasks such as bootstrapping.

Our implementation closely follows the mathematical formulation of VAR models, impulse response functions, Granger causality tests, and residual bootstrap procedures as detailed in Hamilton [1] and Kilian [2]. In this section, we describe how these algorithms are translated into modular, efficient, and parallelizable software.

Variable	Description
LOG_INF_A	Log-transformed weekly influenza A detections ($\log(1 + \text{INF_A})$)
LOG_INF_B	Log-transformed weekly influenza B detections ($\log(1 + \text{INF_B})$)
TEMPERATURE	Average weekly air temperature (degrees Celsius)
DEW_POINT_TEMPERATURE	Average weekly dew point temperature (degrees Celsius)
RELATIVE_HUMIDITY	Average weekly relative humidity (percent), when available
WIND_SPEED	Average weekly wind speed
SEA_LEVEL_PRESSURE	Average weekly sea level pressure
VISIBILITY	Average weekly horizontal visibility
WET_BULB_TEMPERATURE	Average weekly wet-bulb temperature
PRECIPITATION	Weekly average or instantaneous precipitation rate
PRECIPITATION_24_HOUR	Weekly cumulative 24-hour precipitation total
LOG_PRECIPITATION	Log-transformed precipitation, when created during assumption checks
LOG_PRECIPITATION_24_HOUR	Log-transformed 24-hour precipitation, when created
LOG_VISIBILITY	Log-transformed visibility, when created
LOG_WIND_SPEED	Log-transformed wind speed, when created

Table 1: Variables included in the preprocessed country- and strain-specific VAR training datasets.

4.1 Software Architecture

Our Software Architecture consists of 3 layers, each responsible for a different aspect in the data analysis and visualization pipeline.

1. **Data Processing Layer (Python)**
   This layer is responsible for transforming raw data from NOAA and WHO into clean datasets that can be used for training. The scripts perform several key functions:

   - Data Cleaning: Scripts remove inconsistencies, handle missing values in influenza and weather datasets.
   - Data Integration: Weather and epidemiological data are merged, aligned by time and location.
   - Assumption Checking: Before modeling, the scripts apply statistical tests to ensure the data meets the requirements for VAR modeling.
   - Output: The processed data is saved in standardized CSV files, ready for modeling. This allows us to easily change components of each system as long as the csv's maintain their format, each layer can communicate between each other.

2. **VAR Model Layer (Go)**
   This holds our core logic to build the VAR model and perform statistical methods on it. This layer includes:

   - Model Construction: The Go code defines robust data structures (e.g., `TimeSeries`, `ModelSpec`, `ReducedFormVAR`) and methods for fitting Vector Autoregression (VAR) models. It takes

2 parameters as input, one for the country and the opther for the influenza type to use to create the VAR model. This allows us to restrict our analysis to a specific country and viral strain to create a more specialized and accurate model.

- *Advanced Analysis:* After model training, the Go layer provides methods for:
  - Forecasting: Predicting future values of the time series.
  - Impulse Response Functions (IRF): Quantifying the effect of sudden changes to variables to the system over multiple horizons.
  - Granger Causality Analysis: This helps us test for granger causality between our influenza variable and other variables.
  - Bootstrapping: This verifies the output of the model through things such as confidence intervals.
- *Performance:* Go's concurrency features (e.g., goroutines, worker pools) are leveraged for parallel computation.
- *Output:* All results (forecasts, IRFs, causality tables) are written to CSV files to allow the RShiny interface to red them in.

3. **Interface Layer (R Shiny)**

The user interface is built in R Shiny to visualize results.

- User Input: Users select analysis parameters (e.g., country, influenza type) through a web-based UI. This allows users to see how different parameters are significant in different climates for influenza prediction based on weather and how that also varies by influenza strain. (Figure 1)
- Backend Integration: R Shiny invokes the Go binary with the selected parameters, triggering Go to create a model based on the respective parameters, that dictate the dataset used to train the model.
- Data Visualization: The output CSVs from Go are read into R, where `ggplot2` is used to generate plots and gifs based on the analysis the Go code does. These are: a predictive forecast time series (Figure 2), A Granger causality heatmap (Figure 3) and finally a gif for IRF variable shocking over multiple horizons. (Static image of this in Figure 3)

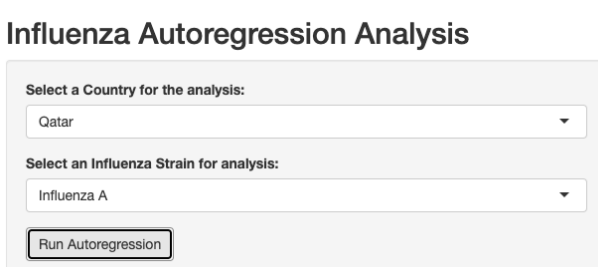

Figure 1: Input Widget

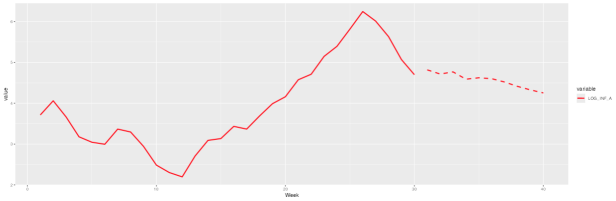

Figure 2: Influenza spread Forecast

Figure 3: Granger Heatmap

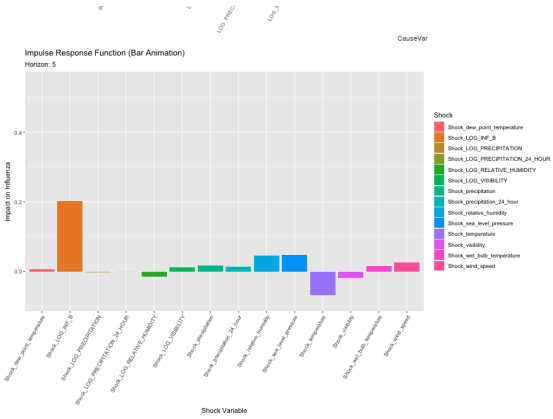

Figure 4: IRF Plot

4.2 VAR Estimation via OLS

At the algorithmic level, VAR estimation follows the stacked regression formulation in section A.1. Given a multivariate time series stored in a `TimeSeries` struct, we construct the regression matrices X and Y by stacking lagged observations and deterministic regressors: each row of X contains p lags of all K variables plus any deterministic terms (constant, trend, or seasonal regressors), and the corresponding row of Y contains the current observation.

This construction is handled in the `OLSEstimator` implementation of the `Estimator` interface. The estimator takes as input:

- a `TimeSeries` object (containing the data matrix and variable names),
- a `ModelSpec` (which encodes the lag order, deterministic terms, and whether exogenous regressors are present),

- and `EstimationOptions` (allowing, for example, future generalizations to GLS or Bayesian methods).

Within `OLSEstimator.Estimate`, we allocate a dense design matrix X using Gonum's `mat.Dense` type, fill it row by row with lagged values, and then solve the multivariate regression

$$\widehat{B} = (X^\top X)^{-1} X^\top Y$$

using Gonum's linear algebra routines. The result is packaged into a `ReducedFormVAR` struct, which stores the estimated lag matrices $A_1, \ldots, A_p$, the deterministic term matrix C, and the residual covariance Σ_u. Because the estimator returns a `ReducedFormVAR` that implements the generic `ReducedForm` interface, all subsequent functionality (forecasting, IRFs, Granger tests, bootstrapping) can operate on that interface rather than on any particular estimation method. This is what makes it easy to swap in a different estimator later without touching the rest of the pipeline.

Deterministic terms are handled transparently via `ModelSpec`. For example, choosing `DetConst` or `DetConstTrend` automatically adds the appropriate columns to X and stores the corresponding coefficients in the `C` field of `ReducedFormVAR`. This keeps the numerical code free of special cases and aligns closely with the mathematical model in eq. (3).

4.3 Impulse Response Computation

Impulse response functions are implemented as a method on the reduced-form model: `(rf *ReducedFormVAR) IRF(horizon, shockIndex)`. Given a fitted model, we first recover the moving-average representation described in section A.3. Algorithmically, this proceeds by:

1. Initializing $\Psi_0 = I_K$.
2. Recursively constructing Ψ_h for $h = 1, \ldots, H$ using the VAR lag matrices $A_1, \ldots, A_p$ and the recursion in eq. (22).
3. Factorizing the residual covariance $\widehat{\Sigma}_u$ using a Cholesky decomposition to obtain a lower-triangular matrix $\widehat{P}$ such that $\widehat{\Sigma}_u = \widehat{P}\widehat{P}^\top$.
4. Forming structural responses $\Theta_h = \Psi_h \widehat{P}$ and extracting columns corresponding to a one-unit shock in a given variable.

The code mirrors this structure closely. The `IRF` method internally builds the sequence of Ψ_h matrices using Gonum dense matrices and simple nested loops. The Cholesky factor is obtained via Gonum's symmetric factorization routines. By confining all of this logic inside the `ReducedFormVAR` methods, the rest of the system only needs to pass a shock index and a horizon and receive a matrix of IRFs, without depending on the specific implementation details of the recursion or factorization.

4.4 Granger Causality Testing

Granger causality tests are implemented in a method `(rf *ReducedFormVAR) GrangerCausalityMatrix(ts *TimeSeries)` that returns a matrix of test results for all ordered pairs of variables. Conceptually, we follow the F-test construction in section A.4: for a given effect variable k and candidate cause j, we compare an unrestricted regression (including all lags of all variables) to a restricted regression in which the p lags of $y_{j,t}$ are excluded from the k-th equation.

Rather than rebuilding design matrices from scratch for every pair, we exploit the structure of the VAR:

- The stacked design matrix for the unrestricted model is already available from the estimation stage and can be re-used.
- Restricted models correspond to imposing linear constraints on the coefficient vector for a given equation; in code, this is implemented by dropping the relevant columns associated with j's lags when forming the equation-specific design matrix.

For each ordered pair (j, k), we compute the unrestricted and restricted residual sums of squares, form the F-statistic as in eq. (34), and compute an associated p-value. These results are stored in a `GrangerCausalityResult` struct and assembled into a matrix for easy downstream use. This matrix representation is convenient both for CSV export and for visualization in the R front-end.

4.5 Residual Bootstrapping for IRFs

Residual bootstrap for IRFs is implemented in the method `(rf *ReducedFormVAR) BootstrapIRF(ts *TimeSeries, opts BootstrapOptions)`. The algorithm follows the scheme in section A.5, but is optimized for performance by parallelizing across bootstrap replications.

At a high level, the procedure is:

1. Fit the VAR once and compute residuals $\widehat{\mathbf{u}}_t$ from the original data.
2. For each bootstrap replication $b = 1, \ldots, B$:
   (a) Resample residual vectors with replacement.
   (b) Simulate a bootstrap time series $\{\mathbf{y}_t^{*(b)}\}$ using the fitted VAR coefficients and the resampled residuals (cf. eq. (38)).
   (c) Re-estimate a `ReducedFormVAR` on the bootstrap sample using the same `Estimator` and `ModelSpec`.
   (d) Compute IRFs for all shocks up to the chosen horizon.
3. Aggregate IRFs across replications to form empirical distributions and quantile-based confidence intervals.

To make this scalable, we parallelize step 2 using a worker-pool pattern:

- We first allocate a slice of per-replication seeds to ensure that each goroutine has its own independent pseudorandom generator, avoiding contention and non-deterministic behavior.
- We then create a job channel that hands out replication indices and a results channel for `irfReplication` objects (each containing all IRFs from a single bootstrap sample).
- A pool of worker goroutines—with size chosen based on the number of CPU cores and the number of replications—reads from the job channel, runs the full simulate–re-fit–IRF pipeline for each bootstrap replication, and sends results back.
- The main goroutine acts as an aggregator, collecting all results and filling a three-dimensional storage structure indexed by shock variable, horizon, and response variable.

Finally, we compute empirical quantiles (e.g., 2.5% and 97.5%) for each entry of the IRF matrix to produce lower and upper confidence bands. These, together with the original point estimates, are returned in an `IRFBootstrapResult` struct for each shock variable.

4.6 Residual Bootstrapping for Granger Causality

Residual bootstrap for Granger causality is implemented in `(rf *ReducedFormVAR) BootstrapGrangerMatrix(ts *TimeSeries, opts GrangerBootstrapOptions)`. The structure parallels IRF bootstrapping but focuses on the F-statistics for Granger tests rather than IRFs.

Each bootstrap replication consists of:

1. Simulating a bootstrap series $\{\mathbf{y}_t^{*(b)}\}$ from the fitted VAR using resampled residuals.
2. Re-estimating a `ReducedFormVAR` on the bootstrap series.
3. Computing the Granger causality matrix for the bootstrap sample.

As with IRFs, these replications are executed in parallel using a worker pool. For each ordered pair (i, j), we compare the observed F-statistic to the empirical distribution of bootstrap F-statistics and compute a bootstrap p-value as in eq. (43). The result is stored in a `GrangerCausalityBootstrapResult`, which augments the original analytic test result with a bootstrap p-value and a boolean flag indicating [**?**] whether the relationship is significant at the chosen level α.

5 Results

To test the model we chose Qatar as our country of interest. We ran the model with both influenza A and influenza B to see how the results change based on the strain of influenza. The output of each type of analysis are discussed below:

5.1 Model Fit and Forecasting Performance

The VAR model was fit to the processed data, and forecasts were generated for key variables. The forecasts show smooth temporal trends and reasonable values, indicating the model captures the main dynamics of the system. For the both influenza A and B forecasts, the model seems to capture the variation in the periodic nature of the graphs respectively but for influenza A the line seems to deviate from historical data (Figure 5 and Figure 6). Both predictions though seem to anticipate a reduction in variation in influenza spread with the predictions showing a line with a lower slope compared to historical data. This may be the case as the end of the flu season often results in less influenza spread.

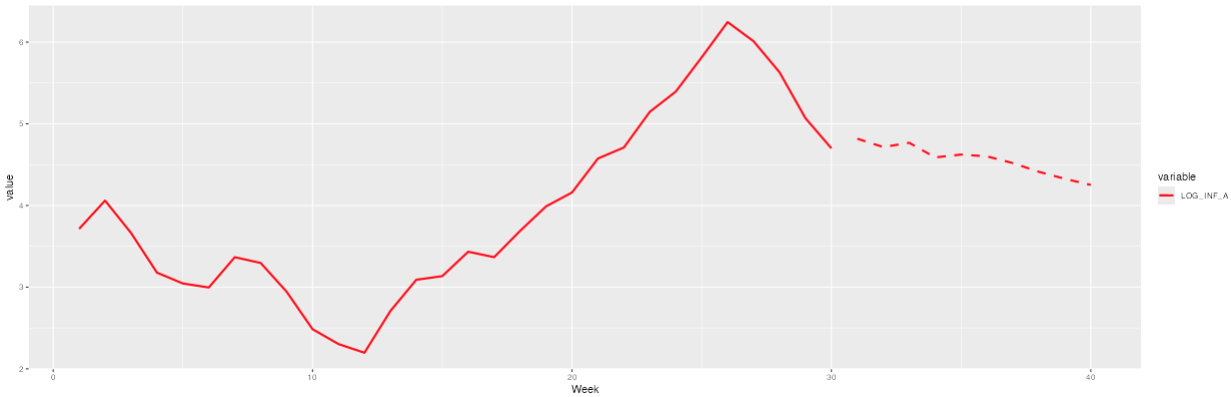

Figure 5: Forecasting Qatar Influenza A

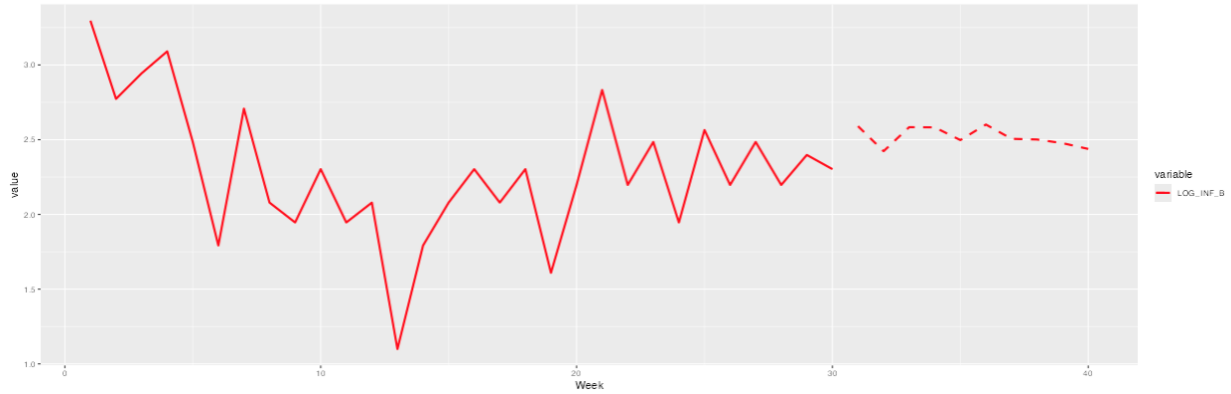

Figure 6: Forecasting Qatar Influenza B

5.2 Impulse Response Analysis

For both strains of influenza, we saw the biggest reaction from the target influenza variable when a few variables were shocked via the Impulse Response functions. Wind Speed and Precipitation consistently showed the greatest initial response and the most sustained responses (Plots not shown

as they are animated gifs). This was true for the regular variables and the log transformed version of them. These results show that our model used these variables the most when trying to make its prediction and therefore they are the most important for the model.

5.3 Granger Causality Results

Finally we have the Granger causality results that indicate which variables provide statistically significant predictive power for others. For influenza A we saw no variables that indicated Granger Causality (Figure 7). This means that out of the variables we supplied the model, none were sufficient enough to make accurate predictions. For influenza B we did see wind speed was a Granger Causal variable (Figure 8). This means its p value was ¡ 0.05 in the analysis and it is a good predictor of influenza B spread in Qatar.

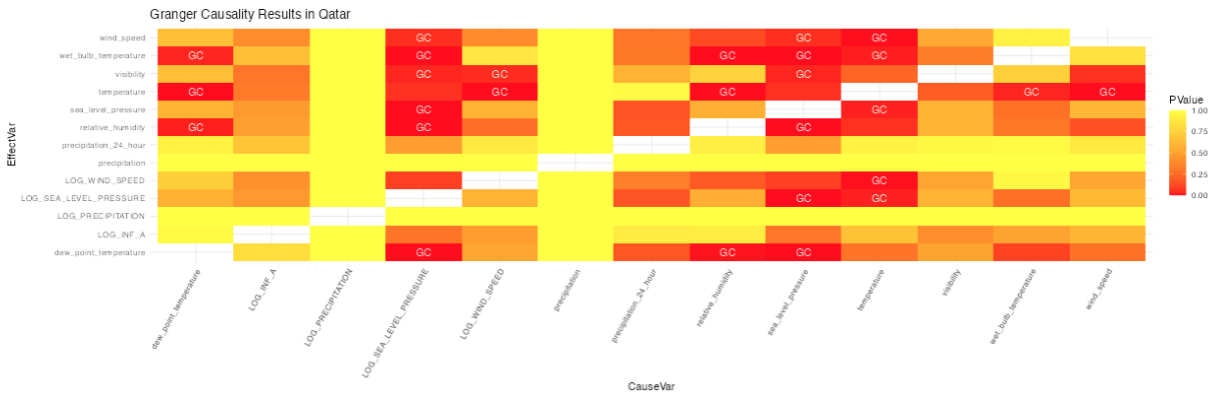

Figure 7: Granger Causality Results for Qatar Influenza A

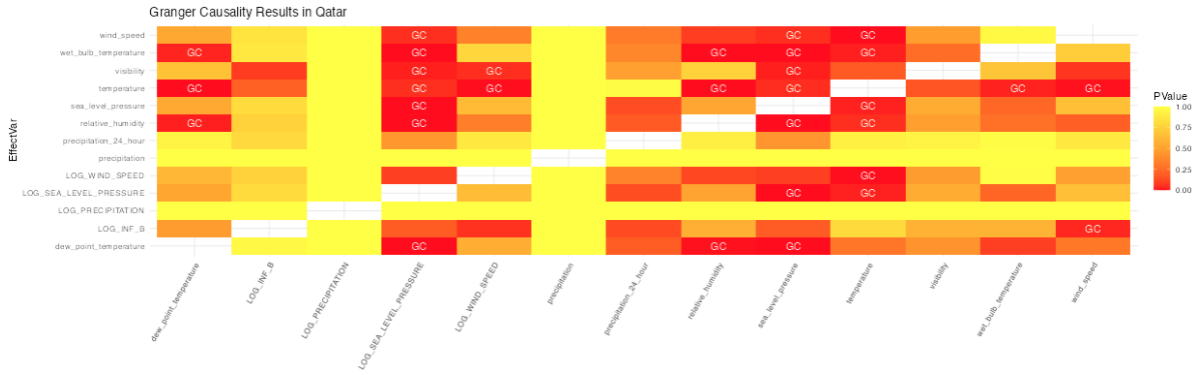

Figure 8: Granger Causality Results for Qatar Influenza B

5.4 Bootstrap Uncertainty Estimates

The bootstrap estimates were done to verify the asymptotic estimates of our model, comparing them to the bootstrap estimates. This is especially useful in case assumptions for the model are not fully met. the results show that while variables such as temperature initially appear to have

significant Granger-causal relationships with other variables under asymptotic tests, these effects are not supported by bootstrap p-values, indicating they may not be robust.

6 Discussion and Conclusion

Our analysis shows that for Qatar, weather variables such as wind and precipitation have the most impact and maintain effect on influenza dynamics, as shown in the IRF analysis. However, the Granger Causality analysis, along with the bootstrap results, showed that some predictive relationships like that for precipitation were not significant enough to make predictions. The one variable that passed our analysis was the wind speed for the spread of Influenza B. This could be a possible predictor as seen in primary literature since wind speed facilitates spread of virus via disruption, temperature and increased dryness . An important consideration here is that we don't expect factors such as wind speed to be a primary predictor for influenza spread when combined with other context dependent variables such as population dynamics, but these results can be used to contribute to the overall forecasting of spread. The modular nature of our code means that we can trivially include more variables for the model to work with, given more data, to create more robust predictions.

7 Acknowledgments

AI Assistance Disclosure. Portions of this report, including drafting, editing, debugging, and code explanation, were assisted by ChatGPT (OpenAI, GPT-5.1 model). All analysis, modeling decisions, and final text were reviewed and approved by the authors.

References

[1] J. D. Hamilton. *Time Series Analysis*. Princeton University Press, 1994.

[2] L. Kilian. Small-sample confidence intervals for impulse response functions. *The Review of Economics and Statistics*, 80(2):218–230, 1998. doi:10.1162/003465398557465.

[3] Lütkepohl, H. (2005). *New Introduction to Multiple Time Series Analysis*. Springer.

[4] Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press.

[5] Shaman, J., & Kohn, M. (2010). Absolute humidity modulates influenza survival, transmission, and seasonality. *Proceedings of the National Academy of Sciences*, 106(9), 3243–3248.

[6] Engle, R. F. (1982). Autoregressive conditional heteroskedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007.

[7] Granger, C. W. J. (1969). Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3), 424–438.

[8] World Health Organization. *FluNet: Global Influenza Surveillance and Response System*. Available at `https://www.who.int/tools/flunet`. Accessed 2024.

[9] National Centers for Environmental Information (NCEI). *Global Historical Climatology Network (GHCN) Hourly Data*. National Oceanic and Atmospheric Administration. Available at `https://www.ncei.noaa.gov/products/land-based-station/global-historical-climatology-network-hourly`. Accessed 2024.

[10] OpenAI. *ChatGPT (GPT-5.1)*, large language model used for writing assistance, editing, and code generation support in this report. https://www.openai.com/chatgpt. Accessed 2025.

[11] A. Peci, A. L. Winter, Y. Li, S. Gnaneshan, J. Liu, S. Mubareka, and J. B. Gubbay. Effects of absolute humidity, relative humidity, temperature, and wind speed on influenza activity in Toronto, Ontario, Canada. *Applied and Environmental Microbiology*, 85(6):e02426–18, 2019. doi:10.1128/AEM.02426-18.

A Supplementary Methods: Mathematical Details

In this appendix we have the mathematical details of the methods used: vector autoregression (VAR), impulse response functions (IRFs), Granger causality tests, and residual bootstrap procedures.

A.1 Vector Autoregression (VAR)

A.1.1 Model specification

Let $\{\mathbf{y}_t\}_{t=1}^T$ be a K-dimensional time series, where $\mathbf{y}_t \in \mathbb{R}^K$ are all the variables (e.g., influenza counts and weather covariates) at time t. A VAR(p) model with intercept can be written as

$$\mathbf{y}_t = \mathbf{c} + \sum_{i=1}^{p} A_i \mathbf{y}_{t-i} + \mathbf{u}_t, \qquad t = 1, \ldots, T, \tag{1}$$

where

- $\mathbf{c} \in \mathbb{R}^K$ is a vector of intercepts (can be modified to add deterministic constants)
- $A_i \in \mathbb{R}^{K \times K}$ are autoregressive coefficient matrices,
- $\mathbf{u}_t \in \mathbb{R}^K$ is the innovation (error) term.

We assume

$$\mathbb{E}[\mathbf{u}_t] = \mathbf{0}, \qquad \mathbb{E}[\mathbf{u}_t \mathbf{u}_s^\top] = \begin{cases} \Sigma_u, & t = s, \\ 0, & t \neq s, \end{cases} \tag{2}$$

so that $\{\mathbf{u}_t\}$ is simply random noise.

A.1.2 Deterministic regressors

We can write eq. (1) with deterministic regressors. Let $\mathbf{d}_t \in \mathbb{R}^M$ be a vector of deterministic terms (e.g., constant, linear trend, seasonal dummies, sine/cosine pairs), and let $B \in \mathbb{R}^{K \times M}$ be its coefficient matrix. Then

$$\mathbf{y}_t = B\mathbf{d}_t + \sum_{i=1}^{p} A_i \mathbf{y}_{t-i} + \mathbf{u}_t. \tag{3}$$

The pure-intercept case corresponds to $\mathbf{d}_t \equiv \mathbf{1}$ and $M = 1$.

A.1.3 Stacked regression form

For estimation, we rewrite the VAR as a multivariate linear regression.

Define the *effective sample size* $T^\star := T - p$ since the first few lags cannot be used. Now we can put all the depdenent variables from $t = p + 1$ to T into a matrix

$$Y = \begin{bmatrix} \mathbf{y}_{p+1}^\top \\ \mathbf{y}_{p+2}^\top \\ \vdots \\ \mathbf{y}_T^\top \end{bmatrix} \in \mathbb{R}^{T^\star \times K}. \tag{4}$$

Next, define the regressor matrix

$$X = \begin{bmatrix} \mathbf{x}_{p+1}^\top \\ \mathbf{x}_{p+2}^\top \\ \vdots \\ \mathbf{x}_T^\top \end{bmatrix} \in \mathbb{R}^{T^\star \times (Kp+M)}, \tag{5}$$

where each row is

$$\mathbf{x}_t^\top = \left[\mathbf{y}_{t-1}^\top, \mathbf{y}_{t-2}^\top, \ldots, \mathbf{y}_{t-p}^\top, \mathbf{d}_t^\top\right] \in \mathbb{R}^{1 \times (Kp+M)}. \tag{6}$$

We also stack the coefficient matrices into a single matrix

$$B = \begin{bmatrix} A_1^\top \\ A_2^\top \\ \vdots \\ A_p^\top \\ B^\top \end{bmatrix} \in \mathbb{R}^{(Kp+M) \times K}. \tag{7}$$

The innovations are stacked as

$$U = \begin{bmatrix} \mathbf{u}_{p+1}^\top \\ \mathbf{u}_{p+2}^\top \\ \vdots \\ \mathbf{u}_T^\top \end{bmatrix} \in \mathbb{R}^{T^\star \times K}. \tag{8}$$

Then eq. (3) for $t = p + 1, \ldots, T$ becomes the matrix equation

$$Y = XB + U. \tag{9}$$

A.2 Estimation via Ordinary Least Squares

The ordinary least squares (OLS) estimator of B minimizes $\|Y - XB\|_F^2$, the sum of squared residuals over all equations:

$$\widehat{B} = \arg\min_B \|Y - XB\|_F^2. \tag{10}$$

Assuming X has full column rank, the solution is

$$\widehat{B} = (X^\top X)^{-1} X^\top Y. \tag{11}$$

The fitted values and residuals are

$$\widehat{Y} = X\widehat{B}, \qquad \widehat{U} = Y - \widehat{Y}. \tag{12}$$

Let $\widehat{\mathbf{u}}_t$ denote the row of $\widehat{U}$ corresponding to time t.

An estimator of the innovation covariance matrix Σ_u is

$$\widehat{\Sigma}_u = \frac{1}{T^\star - (Kp + M)} \widehat{U}^\top \widehat{U} \in \mathbb{R}^{K \times K}, \tag{13}$$

where the denominator uses the residual degrees of freedom. Here M denotes the number of deterministic regressors included in the VAR (e.g., constant, trend, seasonal dummies, or other non-stochastic terms).

Rank-deficient regressors and the Moore–Penrose pseudoinverse

If X does *not* have full column rank, then the matrix $(X^\top X)$ is singular and the closed-form expression (11) is not defined. In this case the OLS estimator is solved using the Moore–Penrose pseudoinverse X^+:

$$\widehat{B} = X^+ Y, \tag{14}$$

which gives us the minimum-norm solution to the least-squares problem $\min_B \|Y - XB\|_F^2$ even if X is rank-deficient. The pseudoinverse is computed via singular value decomposition (SVD).

A.3 Moving-Average Representation and IRFs

A.3.1 Companion form

Let Kp-dimensional stacked state vector

$$\mathbf{Y}_t = \begin{bmatrix} \mathbf{y}_t^\top & \mathbf{y}_{t-1}^\top & \dots & \mathbf{y}_{t-p+1}^\top \end{bmatrix}^\top \in \mathbb{R}^{Kp}. \tag{15}$$

The VAR(p) can be written in first-order form as

$$\mathbf{Y}_t = \mathbf{C} + F\mathbf{Y}_{t-1} + \mathbf{U}_t, \tag{16}$$

where the *companion matrix* $F \in \mathbb{R}^{Kp \times Kp}$ is

$$F = \begin{bmatrix} A_1 & A_2 & \dots & A_{p-1} & A_p \\ I_K & 0 & \dots & 0 & 0 \\ 0 & I_K & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & I_K & 0 \end{bmatrix}, \tag{17}$$

$\mathbf{C} \in \mathbb{R}^{Kp}$ contains the intercepts (and deterministic terms), and

$$\mathbf{U}_t = \begin{bmatrix} \mathbf{u}_t^\top & \mathbf{0}^\top & \dots & \mathbf{0}^\top \end{bmatrix}^\top \in \mathbb{R}^{Kp}. \tag{18}$$

If the VAR is stable, then $(I_{Kp} - F)$ is invertible, and the mean is

$$\mathbb{E}[\mathbf{Y}_t] = (I_{Kp} - F)^{-1}\mathbf{C} =: \boldsymbol{\mu}_Y. \tag{19}$$

$\boldsymbol{\mu}_Y$.

A.3.2 Infinite-order moving-average representation

Under stability, we can rewrite the VAR in *vector moving-average* (VMA(∞)) form:

$$\mathbf{y}_t = \boldsymbol{\mu} + \sum_{h=0}^{\infty} \Psi_h \mathbf{u}_{t-h}, \tag{20}$$

where $\boldsymbol{\mu} \in \mathbb{R}^K$ is the unconditional mean of $\mathbf{y}_t$ and $\Psi_h \in \mathbb{R}^{K \times K}$ are the VMA coefficient matrices. We set

$$\Psi_0 = I_K, \tag{21}$$

and find Ψ_h recursively from the VAR coefficients:

$$\Psi_h = A_1\Psi_{h-1} + A_2\Psi_{h-2} + \dots + A_p\Psi_{h-p}, \qquad h \geq 1, \tag{22}$$

where $\Psi_h = 0$ for $h < 0$.

A.3.3 Structural shocks and identification

The reduced-form innovations $\mathbf{u}_t$ are usually correlated:

$$\mathbb{E}[\mathbf{u}_t\mathbf{u}_t^\top] = \Sigma_u, \qquad \Sigma_u \text{ symmetric positive definite.} \tag{23}$$

To interpret shocks as orthogonal or "structural" shocks, we factor Σ_u as

$$\Sigma_u = PP^\top, \tag{24}$$

where P is a nonsingular $K \times K$ matrix. We can do this using the Cholesky factorization with P lower triangular, which encodes some sort of ordering of the variables. We can then write

$$\mathbf{u}_t = P\boldsymbol{\varepsilon}_t, \qquad \boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, I_K), \tag{25}$$

so that $\boldsymbol{\varepsilon}_t$ has uncorrelated unit-variance. Substituting into eq. (20) gives

$$\mathbf{y}_t = \boldsymbol{\mu} + \sum_{h=0}^{\infty} \Psi_h P \boldsymbol{\varepsilon}_{t-h} = \boldsymbol{\mu} + \sum_{h=0}^{\infty} \Theta_h \boldsymbol{\varepsilon}_{t-h}, \tag{26}$$

where

$$\Theta_h := \Psi_h P \in \mathbb{R}^{K \times K}. \tag{27}$$

A.3.4 Impulse response functions

Consider a one-unit shock to variable j at time t, i.e., $\boldsymbol{\varepsilon}_t = \mathbf{e}_j$ (the j-th basis vector) and $\boldsymbol{\varepsilon}_s = \mathbf{0}$ for $s \neq t$. The effect on $\mathbf{y}_{t+h}$ for horizon $h \geq 0$ is

$$\Delta \mathbf{y}_{t+h} = \Theta_h \mathbf{e}_j = (\text{the } j\text{-th column of } \Theta_h). \tag{28}$$

The *impulse response function* from structural shock j to variable k at horizon h is thus

$$\mathrm{IRF}_{k \leftarrow j}(h) = \big[\Theta_h\big]_{k,j}. \tag{29}$$

In practice, we compute $\widehat{\Psi}_h$ from the estimated VAR coefficients via eq. (22), form $\widehat{\Theta}_h = \widehat{\Psi}_h \widehat{P}$, and then find $\widehat{\mathrm{IRF}}_{k \leftarrow j}(h)$ for $h = 0, \ldots, H$.

A.4 Granger Causality Test Statistic

A.4.1 Definition

Let $y_{k,t}$ and $y_{j,t}$ be two components of the vector $\mathbf{y}_t$. Informally, $y_{j,t}$ *Granger-causes* $y_{k,t}$ if past values of $y_{j,t}$ contain information that helps predict $y_{k,t}$ beyond the information contained in past values of $y_{k,t}$ and other variables.

More precisely, in the context of a VAR(p), we compare two models for the k-th equation:

- **Unrestricted model (U):** includes lags of $y_{j,t}$:

$$y_{k,t} = \beta_0 + \sum_{\ell=1}^{K} \sum_{i=1}^{p} \beta_{\ell,i}^{(U)} y_{\ell,t-i} + \boldsymbol{\gamma}^\top \mathbf{d}_t + u_{k,t}^{(U)}. \tag{30}$$

- **Restricted model (R):** excludes all p lags of $y_{j,t}$, i.e., imposes the linear restrictions $\beta_{j,1}^{(U)} =$

$\cdots = \beta_{j,p}^{(U)} = 0$:

$$y_{k,t} = \beta_0 + \sum_{\ell=1}^{K} \sum_{i=1}^{p} \beta_{\ell,i}^{(R)} y_{\ell,t-i} \quad \text{with } \beta_{j,i}^{(R)} = 0 \text{ for all } i + \boldsymbol{\gamma}^\top \mathbf{d}_t + u_{k,t}^{(R)}. \tag{31}$$

The *null hypothesis* of no Granger causality from $y_{j,t}$ to $y_{k,t}$ is

$$H_0 : \beta_{j,1}^{(U)} = \beta_{j,2}^{(U)} = \cdots = \beta_{j,p}^{(U)} = 0, \tag{32}$$

which says that lags of $y_{j,t}$ do not enter the k-th equation.

A.4.2 F-statistic for linear restrictions

Let $T^\star = T - p$ denote the effective sample size for the regression. For the k-th equation, denote the OLS residuals from the unrestricted and restricted regressions by $\widehat{u}_{k,t}^{(U)}$ and $\widehat{u}_{k,t}^{(R)}$, respectively, and define the residual sums of squares

$$\mathrm{RSS}_U = \sum_{t=p+1}^{T} \left(\widehat{u}_{k,t}^{(U)}\right)^2, \qquad \mathrm{RSS}_R = \sum_{t=p+1}^{T} \left(\widehat{u}_{k,t}^{(R)}\right)^2. \tag{33}$$

Let q be the number of restrictions being tested; here $q = p$ because we restrict p lag coefficients of $y_{j,t}$ in the k-th equation. Let m_U be the number of parameters in the unrestricted equation (including intercept and deterministic terms). The usual F-statistic is

$$F = \frac{(\mathrm{RSS}_R - \mathrm{RSS}_U)/q}{\mathrm{RSS}_U/(T^\star - m_U)}. \tag{34}$$

Under H_0 and standard regularity conditions, F has an F-distribution with $(q,\, T^\star - m_U)$ degrees of freedom:

$$F \sim F_{q,\, T^\star - m_U}. \tag{35}$$

We can then obtain a p-value by comparing the observed F to this distribution, and reject the null of no Granger causality if the p-value is less than a chosen significance level (e.g., $\alpha = 0.05$).

In matrix form, multiple Granger tests can be performed simultaneously using the estimated coefficient matrices $\widehat{A}_i$ and the estimated innovation covariance $\widehat{\Sigma}_u$, but in our implementation we apply eq. (34) equation-by-equation for the sake of interpretability.

A.5 Residual Bootstrap for IRFs

Analytic IRF confidence intervals are derived under assumptions such as correct VAR specification, Gaussian and homoskedastic errors, and large-sample asymptotics. In our application, these assumptions are not fully satisfied. To obtain inference that is robust to such deviations, we use a residual bootstrap, which approximates the sampling variability of the IRFs directly from the

observed data without relying on these parametric assumptions.

A.5.1 Algorithm

Let $\{\mathbf{y}_t\}_{t=1}^T$ be the observed series, and let $\widehat{A}_1, \ldots, \widehat{A}_p$, $\widehat{B}$, and $\widehat{\Sigma}_u$ be the parameters estimated by OLS as described in section A.1. Let $\widehat{\mathbf{u}}_t$ be the corresponding residuals for $t = p+1, \ldots, T$.

Now consider B bootstrap replications. For replication $b = 1, \ldots, B$:

(B1) Resample residuals. Construct a bootstrap residual sequence $\{\mathbf{u}_t^{*(b)}\}_{t=p+1}^T$ by drawing with replacement from the set of centered residuals $\{\widehat{\mathbf{u}}_t - \bar{\mathbf{u}} : t = p+1, \ldots, T\}$, where

$$\bar{\mathbf{u}} = \frac{1}{T^\star} \sum_{t=p+1}^{T} \widehat{\mathbf{u}}_t. \tag{36}$$

Centering enforces zero mean in the bootstrap innovations. Because the distribution of the residuals is taken from the data rather than assumed, this step avoids parametric assumptions such as normality.

(B2) Initialize bootstrap series. Choose starting values for the bootstrap series $\mathbf{y}_1^{*(b)}, \ldots, \mathbf{y}_p^{*(b)}$. Weset

$$\mathbf{y}_t^{*(b)} = \mathbf{y}_t, \qquad t = 1, \ldots, p, \tag{37}$$

Using the observed history ensures that each bootstrap sample shares the same starting conditions as the original dataset.

(B3) Simulate bootstrap series. For $t = p+1, \ldots, T$, generate

$$\mathbf{y}_t^{*(b)} = \widehat{B}\,\mathbf{d}_t + \sum_{i=1}^{p} \widehat{A}_i \mathbf{y}_{t-i}^{*(b)} + \mathbf{u}_t^{*(b)}. \tag{38}$$

This produces a bootstrap sample $\{\mathbf{y}_t^{*(b)}\}_{t=1}^T$.

(B4) Re-fit VAR to bootstrap sample. Estimate a VAR(p) on $\{\mathbf{y}_t^{*(b)}\}$ in the exact same way as for the original data, obtaining $\widehat{A}_i^{*(b)}$, $\widehat{B}^{*(b)}$ and $\widehat{\Sigma}_u^{*(b)}$.

(B5) Compute bootstrap IRFs. From $\{\widehat{A}_i^{*(b)}\}$ and $\widehat{\Sigma}_u^{*(b)}$, compute the bootstrap VMA coefficients $\widehat{\Psi}_h^{*(b)}$ for $h = 0, \ldots, H$ using eq. (22), obtain a bootstrap Cholesky factor $\widehat{P}^{*(b)}$ such that $\widehat{\Sigma}_u^{*(b)} = \widehat{P}^{*(b)}\widehat{P}^{*(b)\top}$, form $\widehat{\Theta}_h^{*(b)} = \widehat{\Psi}_h^{*(b)}\widehat{P}^{*(b)}$, and extract bootstrap IRFs

$$\widehat{\mathrm{IRF}}_{k\leftarrow j}^{*(b)}(h) = \left[\widehat{\Theta}_h^{*(b)}\right]_{k,j}, \qquad h = 0, \ldots, H. \tag{39}$$

After B replications, we get distributions for each IRF entry and horizon:

$$\left\{\widehat{\mathrm{IRF}}_{k\leftarrow j}^{*(b)}(h)\right\}_{b=1}^{B}. \tag{40}$$

A $(1-\alpha)$ confidence interval at horizon h can be gotten using quantiles, e.g. the percentile method:

$$\Big[q_{\alpha/2},\ q_{1-\alpha/2}\Big], \tag{41}$$

where q_τ is the τ-quantile of the bootstrap sample. Because these intervals are derived from the empirical distribution rather than asymptotic approximations, they remain valid even when standard VAR assumptions (such as normality or homoskedasticity) are violated.

A.6 Residual Bootstrap for Granger Causality

We also use a residual bootstrap to obtain empirical distributions and p-values for the Granger causality F-statistics in eq. (34). The goal is to simulate many bootstrap series using the fitted VAR model, re-fit the model to each bootstrap series, recompute the F-statistics, and compare the observed statistic to this bootstrap distribution.

Let F_{obs} be the observed F-statistic for a given Granger test (e.g., from variable j to k). Using the same bootstrap series $\{\mathbf{y}_t^{*(b)}\}_{t=1}^T$ generated in section A.5, we do:

(G1) Re-fit VAR to bootstrap series. For replication b, estimate the VAR(p) on $\{\mathbf{y}_t^{*(b)}\}$, obtaining $\{\widehat{A}_i^{*(b)}\}$ and $\widehat{B}^{*(b)}$.

(G2) Compute bootstrap F-statistic. For the Granger relationship of interest (e.g., $j \to k$), re-estimate the unrestricted and restricted versions of the k-th equation on the bootstrap series and compute $F^{*(b)}$ using the same formula as in eq. (34).

This yields bootstrap statistics

$$\{F^{*(b)}\}_{b=1}^B. \tag{42}$$

A *bootstrap p-value* for the observed F_{obs} is then

$$\widehat{p}_{\text{boot}} = \frac{1 + \#\{b : F^{*(b)} \geq F_{\text{obs}}\}}{B+1}. \tag{43}$$

If $\widehat{p}_{\text{boot}} < \alpha$ (e.g., $\alpha = 0.05$), we regard the Granger relationship as statistically significant at level α, based on the bootstrap approximation to the finite-sample distribution.

The bootstrap framework in sections A.5 and A.6 thus provides a unified way to quantify uncertainty in both impulse responses and Granger causality tests, using the same core VAR model and residual structure.

B Author Contributions

Dylan Setiawan

- Acquired, cleaned, and standardized NOAA weather-station data for each country for use in the influenza training dataset.
- Designed and implemented the core VAR infrastructure in Go, including data abstractions, OLS estimation, forecasting routines, and IRF computation, spanning `datatypes.go`, `functions.go`, `io.go`, and `main.go`.
- Implemented the parallelized residual bootstrap framework for uncertainty quantification in IRFs and Granger causality, using worker-pool concurrency and empirical quantile aggregation.
- Wrote the mathematical appendix, technical background and major sections of the Algorithms & Implementation (Sections 4.2–4.6).

Rohan Adla

- High level Go code design and structs, developed Go functions to process and output results in CSV's and run IRF's, developed main.go. Created pipeline to allow the user to select parameters and to ingest training data, train model and output data via R
- Developed R Shiny front end, and R server code to compile Go and output results in graphs made using ggplot via images and gifs
- Wrote the Software Architecture and Results sections

Shreyan Nalwad

- Designed and implemented the Granger causality analysis module, that includes restricted and unrestricted modeling, to systematically compute F-statistics, p-values, and direction-specific "Granger-causes" decisions for all ordered pairs of variables.
- Worked with data quality and ensuring data assumption requirements are met, by ranking candidate countries by missing values, selecting higher-quality surveillance and weather series, and applying interpolation based gap filling to support downstream VAR modeling.
- Developed unit tests for the Go backend to validate correctness and robustness of the code.
- Wrote the Data and Preprocessing section (Section 3).

Arrio Gonsalves

- Processed and cleaned FluNet surveillance data, including exploratory analysis of distributions and evaluation of key assumptions such as linearity and homoscedasticity.
- Integrated FluNet and NOAA datasets by aligning records on matching year–week indices, applied log and differencing transformations to influenza A and influenza B series, and prepared separate datasets for strain-specific analysis.
- Authored the Introduction (Section 1), motivating the biological problem and framing the computational objectives of the project.